



Department of Computer Science
Faculty of Computing
UNIVERSITI TEKNOLOGI MALAYSIA

SUBJECT NAME:	COMPUTER ORGANIZATION AND ARCHITECTURE				
SUBJECT CODE:	SECR 1033				
SEMESTER:	2 – 2023/2024				
LAB TITLE:	Lab 2: Arithmetic Equations & Operations				
STUDENT INFO:	Execute the lab in group of two. <table border="1"> <thead> <tr> <th>Student 1</th><th>Student 2</th></tr> </thead> <tbody> <tr> <td>No. 1, 3, 5</td><td>No 2, 4, 6</td></tr> </tbody> </table> Name 1: Nabil Aflah Boo Bt Mohd Yosuf Boo Yong Chong Metric No: A23CS0252 Link for Video Demo: https://drive.google.com/drive/folders/1D9ndZ_6NrRkQ4ieQiv-mqKF_1dB1gcpC?usp=sharing Name 2: Nurul Adriana Binti Kamal Jefri Metric No: A23CS0258 Link for Video Demo: https://drive.google.com/drive/folders/1dDTkOXd-1uT7d3rHvx05AwKkfR9Ag9ff?usp=drive_link	Student 1	Student 2	No. 1, 3, 5	No 2, 4, 6
Student 1	Student 2				
No. 1, 3, 5	No 2, 4, 6				
SUBMISSION DATE & ITEMS:	Duration of submission: - 2 weeks Submission items in e-learning: -				

	1. Lab 2 exercise sheet/file (in .pdf), with the links for demo video 5 - 10min, on the cover page. 2. The assembly programs (in.asm).
--	--

MARKS:

Arithmetic Equation Coding in Assembly Language

Q1. Execute the program below. Determine output of the program by inspecting the content of the related registers.

- Fill in Table 1 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.
- Paste the screenshot of all registers' content after each LINE is executed.

```
INCLUDE Irvine32.inc
.data
var1 word 1
var2 word 9

.code
main PROC
    mov ax, var1      ; LINE1
    mov bx, var2      ; LINE2
    xchg ax, bx       ; LINE3
    mov var1, ax      ; LINE4
    mov var2, bx      ; LINE5
    call DumpRegs
    exit
main ENDP
END main
```

Answer Q1

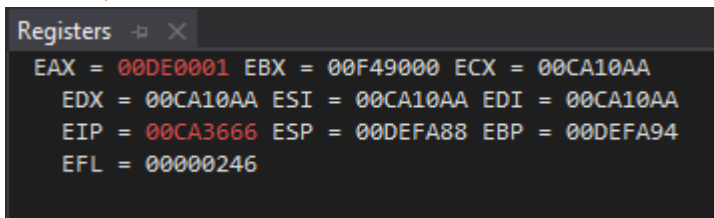
- Fill (Write) in the contents for the related register in each line:

Table 1

LINE1	AX = 0001h var1 = 0001h	Move the value of var1 (1d) into register AX
LINE2	BX = 0009h var2 = 0009h	Move the value of var2 into register BX
LINE3	AX = 0009h BX = 0001h	Exchange the register BX with register AX
LINE4	AX = 0009h var1 = 0009h	Move the value of register AX into var1
LINE5	BX = 0001h var2 = 0001h	Move the value of register BX into var2

- Paste here screenshot of all registers' content after each LINE is executed:

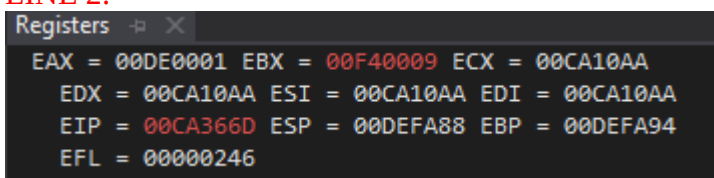
LINE 1:



Registers

EAX = 00DE0001 EBX = 00F49000 ECX = 00CA10AA
EDX = 00CA10AA ESI = 00CA10AA EDI = 00CA10AA
EIP = 00CA3666 ESP = 00DEFA88 EBP = 00DEFA94
EFL = 00000246

LINE 2:



Registers

EAX = 00DE0001 EBX = 00F40009 ECX = 00CA10AA
EDX = 00CA10AA ESI = 00CA10AA EDI = 00CA10AA
EIP = 00CA366D ESP = 00DEFA88 EBP = 00DEFA94
EFL = 00000246

LINE 3:

```
Registers  ▢ ✕  
EAX = 00DE0009 EBX = 00F40001 ECX = 00CA10AA  
EDX = 00CA10AA ESI = 00CA10AA EDI = 00CA10AA  
EIP = 00CA366F ESP = 00DEFA88 EBP = 00DEFA94  
EFL = 00000246
```

LINE 4:

```
Registers  ▢ ✕  
EAX = 00DE0009 EBX = 00F40001 ECX = 00CA10AA  
EDX = 00CA10AA ESI = 00CA10AA EDI = 00CA10AA  
EIP = 00CA3675 ESP = 00DEFA88 EBP = 00DEFA94  
EFL = 00000246
```

LINE 5:

```
Registers  ▢ ✕  
EAX = 00DE0009 EBX = 00F40001 ECX = 00CA10AA  
EDX = 00CA10AA ESI = 00CA10AA EDI = 00CA10AA  
EIP = 00CA367C ESP = 00DEFA88 EBP = 00DEFA94  
EFL = 00000246
```

Q2. Execute the program below. Determine output of the program by inspecting the content of the related registers and watches.

- Fill in Table 2 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.
- Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $Rval = (-Xval + (Yval - Zval)) + 1$

```
include irvine32.inc

.data
Rval DWORD ?
Xval DWORD 26
Yval DWORD 30
Zval DWORD 40

.code
main proc
    mov eax,Xval      ; LINE1
    neg eax           ; LINE2
    mov ebx,Yval      ; LINE3
    sub ebx,Zval      ; LINE4
    add eax,ebx       ; LINE5
    inc eax           ; LINE6
    mov Rval,eax      ; LINE7
    exit
main endp
end main
```

Answer Q2

- Fill (Write) in the contents for the related register in each line:

Table 2

LINE1	EAX = 0000001Ah Xval = 0000001Ah	Move the value of Xval (26d) into register EAX
LINE2	EAX = FFFFFFFE6h	Negate the value in EAX
LINE3	EBX = 0000001Eh Yval = 0000001Eh	Move the value of Yval (30d) into register EBX
LINE4	EBX = FFFFFFFF6h Zval = 00000028h	Subtract the value of Zval (40d) from EBX
LINE5	EAX = FFFFFFFDCh EBX = FFFFFFFF6h	Add the value of EBX to
LINE6	EAX = FFFFFFFDDh	Increment the value in EAX
LINE7	EAX = FFFFFFFDDh Rval = FFFFFFFDDh	Move the value in EAX to Rval

- Paste here screenshot of all registers' content after each LINE is executed:

LINE 1:

```
Registers
EAX = 0133FAFC EBX = 0111F000 ECX = 00201005 EDX = 00201005
ESI = 00201005 EDI = 00201005 EIP = 00201010 ESP = 0133FAA8
EBP = 0133FAB4 EFL = 00000246

0x00204004 = 0000001A
```

LINE 2:

```
Registers
EAX = 0000001A EBX = 0111F000 ECX = 00201005 EDX = 00201005
ESI = 00201005 EDI = 00201005 EIP = 00201015 ESP = 0133FAA8
EBP = 0133FAB4 EFL = 00000246
```

LINE 3:

```
Registers
EAX = FFFFFFFE6 EBX = 0111F000 ECX = 00201005 EDX = 00201005
ESI = 00201005 EDI = 00201005 EIP = 00201017 ESP = 0133FAA8
EBP = 0133FAB4 EFL = 00000293

0x00204008 = 0000001E
```

LINE 4:

```
Registers
EAX = FFFFFFFE6 EBX = 0000001E ECX = 00201005 EDX = 00201005
ESI = 00201005 EDI = 00201005 EIP = 0020101D ESP = 0133FAA8
EBP = 0133FAB4 EFL = 00000293

0x0020400C = 00000028
```

LINE 5:

```
Registers
EAX = FFFFFFFE EBX = FFFFFFF6 ECX = 00201005 EDX = 00201005
ESI = 00201005 EDI = 00201005 EIP = 00201023 ESP = 0133FAA8
EBP = 0133FAB4 EFL = 00000287
```

LINE 6:

```
Registers
EAX = FFFFFFFC EBX = FFFFFFF6 ECX = 00201005 EDX = 00201005
ESI = 00201005 EDI = 00201005 EIP = 00201025 ESP = 0133FAA8
EBP = 0133FAB4 EFL = 00000283
```

LINE 7:

```
Registers
EAX = FFFFFFFD EBX = FFFFFFF6 ECX = 00201005 EDX = 00201005
ESI = 00201005 EDI = 00201005 EIP = 00201026 ESP = 0133FAA8
EBP = 0133FAB4 EFL = 00000287

0x00204000 = 00000000
```

Q3. Execute the program below. Determine output of the program by inspecting the content of the related registers.

- Fill in Table 3 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.
- Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $\text{var4} = [(\text{var1} * \text{var2}) + \text{var3}] - 1$

```
include irvine32.inc

.data
var1 DWORD 5
var2 DWORD 10
var3 DWORD 20
var4 DWORD ?

.code
main proc
    mov eax, var1        ; LINE1
    mul var2              ; LINE2
    add eax, var3         ; LINE3
    dec eax               ; LINE4
    exit
main endp
end main
```

Answer Q3

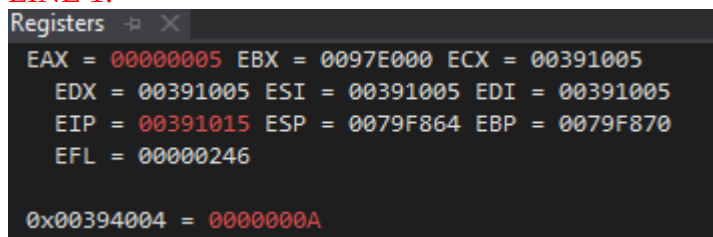
- Fill (Write) in the contents for the related register in each line:

Table 3

LINE1	EAX = 00000005h var1 = 00000005h	Move the value of var1 (5d) into register EAX
LINE2	EAX = 00000032h var2 = 0000000Ah	Multiply with the value of var2
LINE3	EAX = 00000046h var3 = 00000014h	Add the value of var3 with register EAX
LINE4	EAX = 00000045h var4 = 00000069	Decrement with the register EAX

- Paste here screenshot of all registers' content after each LINE is executed:

LINE 1:



```
Registers
EAX = 00000005 EBX = 0097E000 ECX = 00391005
EDX = 00391005 ESI = 00391005 EDI = 00391005
EIP = 00391015 ESP = 0079F864 EBP = 0079F870
EFL = 00000246

0x00394004 = 0000000A
```

LINE 2:

Registers

EAX = 00000032 EBX = 0097E000 ECX = 00391005
EDX = 00000000 ESI = 00391005 EDI = 00391005
EIP = 00391018 ESP = 0079F864 EBP = 0079F870
EFL = 00000202

0x00394008 = 00000014

LINE 3:

Registers

EAX = 00000046 EBX = 0097E000 ECX = 00391005
EDX = 00000000 ESI = 00391005 EDI = 00391005
EIP = 00391021 ESP = 0079F864 EBP = 0079F870
EFL = 00000202

LINE 4:

Registers

EAX = 00000045 EBX = 0097E000 ECX = 00391005
EDX = 00000000 ESI = 00391005 EDI = 00391005
EIP = 00391022 ESP = 0079F864 EBP = 0079F870
EFL = 00000202

Q4. Execute the program below. Determine output of the program by inspecting the content of the related registers.

- Fill in Table 4 with the content of each register or variable on every LINE, in Hexadecimal (as per the output). Please complete the comments for every LINE.
- Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $\text{var4} = (\text{var1} * 5) / (\text{var2} - 3)$

```
include irvine32.inc
.data
    var1 WORD 40
    var2 WORD 10
    var4 WORD ?
.code
main proc
    mov ax,var1      ; LINE1
    mov bx,5         ; LINE2
    mul bx           ; LINE3
    mov bx,var2       ; LINE4
    sub bx,3         ; LINE5
    div bx           ; LINE6
    mov var4,ax       ; LINE7
    exit
main endp
end main
```

Answer Q4

- Fill (Write) in the contents for the related register in each line:

Table 4

LINE1	AX = 0028h var1 = 0028h	Move the value of var1 (40d) into register AX
LINE2	BX = 0005h	Move integer value (5) into register BX
LINE3	AX = 00C8h BX = 0005h	Multiply register AX to register BX, and store into register AX
LINE4	BX = 000Ah var2 = 000Ah	Move integer value (5) into register BX
LINE5	BX = 0007h	Subtract integer value (3) from EBX
LINE6	AX = 002Ch BX = 0007h DX = 0007h	Divide DX by BX, quotient in AX, remainder in DX
LINE7	AX = 002Ch var4 = 002Ch	Move the value in register AX to var4

- Paste here screenshot of all registers' content after each LINE is executed:

LINE 1:

```
Registers
EAX = 00B3FE4C EBX = 008FF000 ECX = 00CC1005 EDX = 00CC1005
ESI = 00CC1005 EDI = 00CC1005 EIP = 00CC1010 ESP = 00B3FDF8
EBP = 00B3FE04 EFL = 00000246
```

100 %

LINE 2:

```
Registers
EAX = 00B30028 EBX = 008FF000 ECX = 00CC1005 EDX = 00CC1005
ESI = 00CC1005 EDI = 00CC1005 EIP = 00CC1016 ESP = 00B3FDF8
EBP = 00B3FE04 EFL = 00000246
```

100 %

LINE 3:

```
Registers
EAX = 00B30028 EBX = 008F0005 ECX = 00CC1005 EDX = 00CC1005
ESI = 00CC1005 EDI = 00CC1005 EIP = 00CC101A ESP = 00B3FDF8
EBP = 00B3FE04 EFL = 00000246
```

100 %

LINE 4:

```
Registers
EAX = 00B300C8 EBX = 008F0005 ECX = 00CC1005 EDX = 00CC0000
ESI = 00CC1005 EDI = 00CC1005 EIP = 00CC101D ESP = 00B3FDF8
EBP = 00B3FE04 EFL = 00000202
```

100 %

LINE 5:

```
Registers
EAX = 00B300C8 EBX = 008F000A ECX = 00CC1005 EDX = 00CC0000
ESI = 00CC1005 EDI = 00CC1005 EIP = 00CC1024 ESP = 00B3FDF8
EBP = 00B3FE04 EFL = 00000202
```

LINE 6:

```
Registers
EAX = 00B300C8 EBX = 008F0007 ECX = 00CC1005 EDX = 00CC0000
ESI = 00CC1005 EDI = 00CC1005 EIP = 00CC1028 ESP = 00B3FDF8
EBP = 00B3FE04 EFL = 00000202
```

LINE 7:

```
Registers
EAX = 00B3001C EBX = 008F0007 ECX = 00CC1005 EDX = 00CC0004
ESI = 00CC1005 EDI = 00CC1005 EIP = 00CC102B ESP = 00B3FDF8
EBP = 00B3FE04 EFL = 00000202
```

Short Notes for MUL CX and DIV BL:

MUL CX

- a. MUL always uses AX (or its extended versions EAX or RAX) as the implicit destination register.
- b. The operand size determines the size of the result:
 - i. Byte-sized operand: Result in AX
 - ii. Word-sized operand: Result in DX:AX
 - iii. Doubleword-sized operand (32-bit mode): Result in EDX:EAX
 - iv. Quadword-sized operand (64-bit mode): Result in RDX:RAX
- c. The upper half of the result (DX or EDX or RDX) holds any overflow bits.
- d. The Carry Flag (CF) is set if the upper half of the product is non-zero.

DIV BL

- a. DIV always uses the DX:AX or EDX:EAX pair as the implicit dividend register.
 - b. The divisor is specified as the operand of the DIV instruction.
 - c. The quotient is stored in AX (for 16-bit division) or EAX (for 32-bit division).
 - d. The remainder is stored in DX.
 - e. Clear DX (or EDX for 32-bit division) before division to ensure a correct 16-bit or 32-bit dividend.
 - f. If the divisor is 0, a division error occurs.
 - g. The Overflow Flag (OF) is set if the quotient is too large to fit in the destination register.
-

Q5. Given the following instructions as is Code Snippet 1.

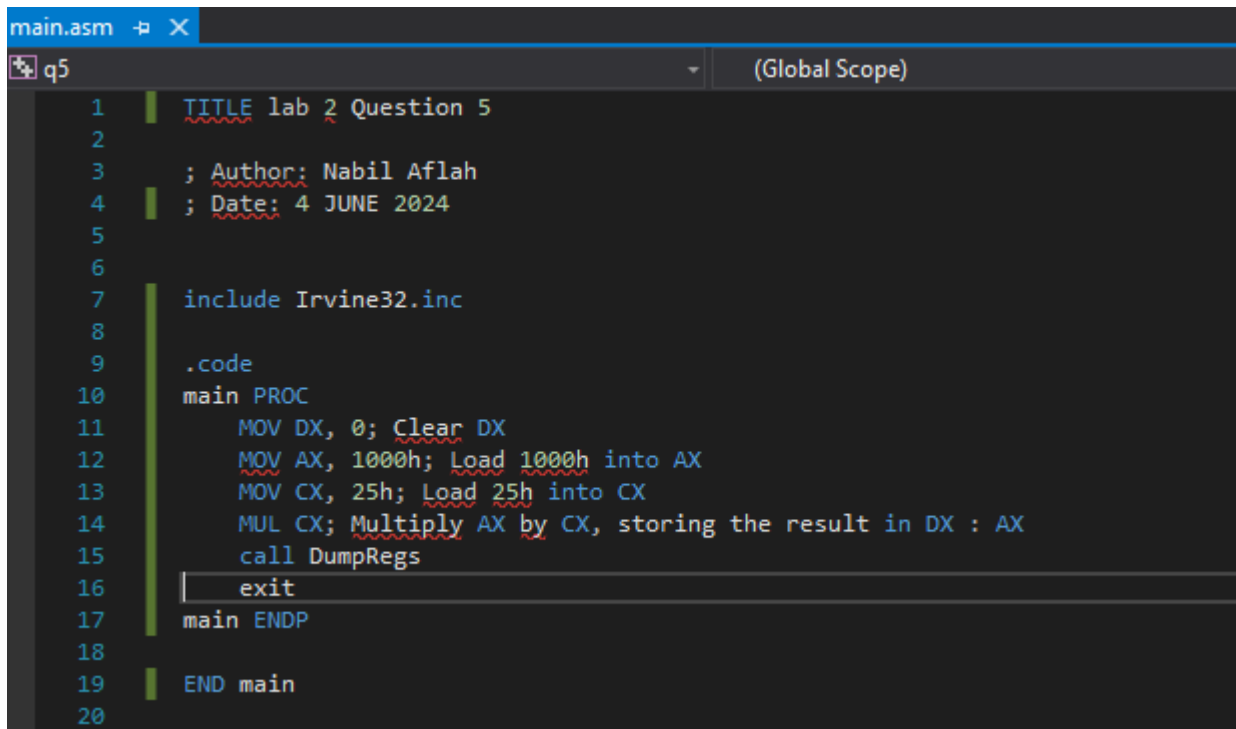
- a) Write a full program to execute the Code Snippet 1.
- b) What are the contents of the related registers after Code Snippet 1 is executed? Paste the screenshot of DumpReg.

; Code Snippet 1 (MUL CX)

```
MOV DX, 0          ; Clear DX
MOV AX, 1000h       ; Load 1000h into AX
MOV CX, 25h         ; Load 25h into CX
MUL CX              ; Multiply AX by CX, storing the result in DX:AX
```

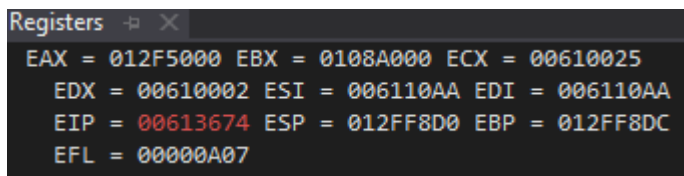
Answer Q5

- a) Screenshot of full program (.asm):



```
main.asm -p X
q5 (Global Scope)
1  TITLE lab 2 Question 5
2
3  ; Author: Nabil Aflah
4  ; Date: 4 JUNE 2024
5
6
7  include Irvine32.inc
8
9  .code
10 main PROC
11     MOV DX, 0; Clear DX
12     MOV AX, 1000h; Load 1000h into AX
13     MOV CX, 25h; Load 25h into CX
14     MUL CX; Multiply AX by CX, storing the result in DX : AX
15     call DumpRegs
16     exit
17 main ENDP
18
19 END main
20
```

- b) Paste here the screenshot of the final registers' content (DumpReg):



```
Registers -p X
EAX = 012F5000 EBX = 0108A000 ECX = 00610025
EDX = 00610002 ESI = 006110AA EDI = 006110AA
EIP = 00613674 ESP = 012FF8D0 EBP = 012FF8DC
EFL = 00000A07
```

Q6. Given the following instructions as is Code Snippet 2.

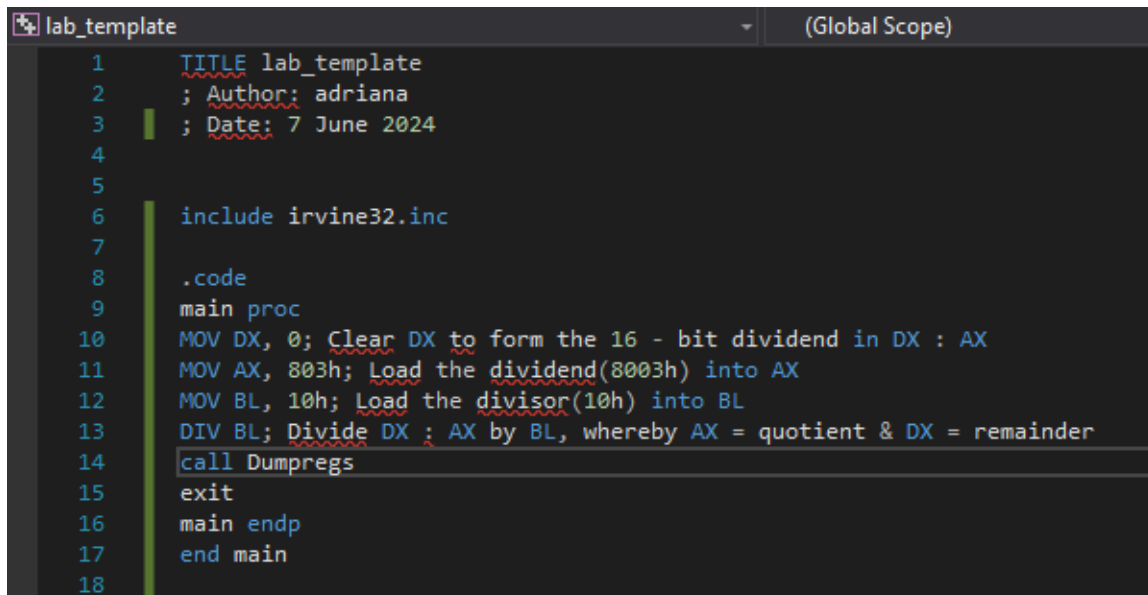
- Write a full program to execute the Code Snippet 2.
- What are the contents of the related registers after Code Snippet 2 is executed? Paste the screenshot of DumpReg.

; Code Snippet 2 (DIV BL)

```
MOV DX, 0           ; Clear DX to form the 16-bit dividend in DX:AX
MOV AX, 803h        ; Load the dividend (8003h) into AX
MOV BL, 10h         ; Load the divisor (10h) into BL
DIV BL              ; Divide DX:AX by BL, whereby AX=quotient & DX=remainder
```

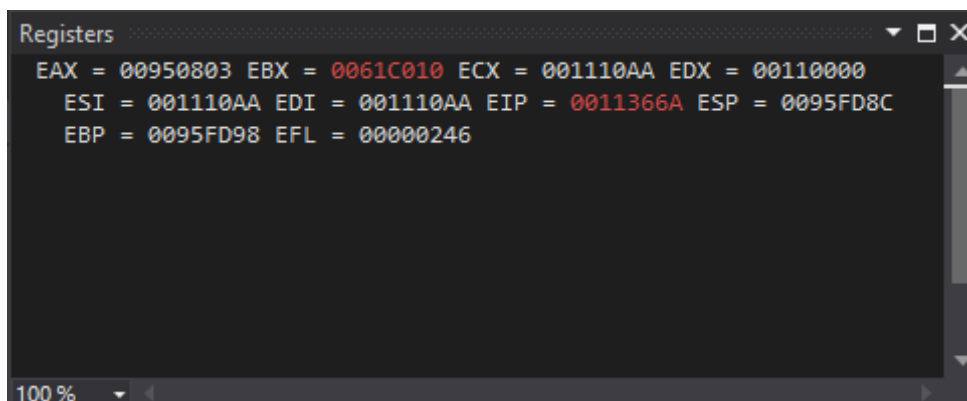
Answer Q6

- Screenshot of full program (.asm):



```
lab_template (Global Scope)
1  TITLE lab_template
2  ; Author: adriana
3  ; Date: 7 June 2024
4
5
6  include irvine32.inc
7
8  .code
9  main proc
10 MOV DX, 0; Clear DX to form the 16 - bit dividend in DX : AX
11 MOV AX, 803h; Load the dividend(8003h) into AX
12 MOV BL, 10h; Load the divisor(10h) into BL
13 DIV BL; Divide DX : AX by BL, whereby AX = quotient & DX = remainder
14 call Dumpregs
15 exit
16 main endp
17 end main
18
```

- Paste here the screenshot of the final registers' content (DumpReg):



```
Registers
EAX = 00950803 EBX = 0061C010 ECX = 001110AA EDX = 00110000
ESI = 001110AA EDI = 001110AA EIP = 0011366A ESP = 0095FD8C
EBP = 0095FD98 EFL = 00000246
100 %
```